



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM

HỌC PHẦN LẬP TRÌNH MẠNG (NETWORK PROGRAMMING) • CHƯƠNG 2

Lập trình Socket với Python

Khoa Toán - Tin • Năm học 2026 – 2027



Mục tiêu Chương 2



Khái niệm socket

Giải thích socket là gì, họ địa chỉ và kiểu socket; phân biệt socket lắng nghe với socket phiên.

Mô-đun socket

Sử dụng đúng socket(), bind(), listen(), accept(), connect(), send(), recv(), close().

Chương trình đầu tay

Viết và chạy được cặp echo server – echo client TCP trên cùng máy và trên hai máy khác nhau.

Phân giải tên miền

Dùng gethostbyname(), getaddrinfo(), gethostbyaddr() để tra cứu tên máy và địa chỉ IP.

Gỡ lỗi và công cụ

Nhận diện ConnectionRefusedError, TimeoutError, gaierror; kiểm tra bằng netstat, ncat, Wireshark.

Làm việc nhóm

Cùng nhóm cài đặt bước đăng nhập của giao thức chat đã đặc tả ở Chương 1 (CLO 5).



Nội dung Chương 2



2.1

Giới thiệu Socket

Slides 4–13

2.2

Mô-đun socket trong Python

Slides 14–30

2.3

Phân giải tên miền DNS

Slides 31–40

Ba mục theo đề cương học phần • 3 tiết lý thuyết + 1 tiết thực hành • CLO 2, CLO 5



2.1

Giới thiệu Socket

Socket là gì; bộ năm định danh kết nối; họ địa chỉ và kiểu socket; trình tự lời gọi TCP và UDP; socket lắng nghe và socket phiên; chặn, bộ đệm, chọn cổng





Socket là điểm cuối (endpoint) của một kênh liên lạc hai chiều giữa hai tiến trình. Hệ điều hành tạo và quản lý socket, ứng dụng sử dụng nó thông qua một tập hàm gọi là socket API.

- ▶ **Nguồn gốc:** ra đời trên BSD UNIX 4.2 (1983), nay là chuẩn chung — POSIX sockets trên Linux/macOS, Winsock trên Windows.
- ▶ **Giống như tệp:** hệ điều hành trả về một mô tả (file descriptor / handle); ứng dụng mở, ghi, đọc rồi đóng.
- ▶ **Hai chiều:** cùng một socket vừa gửi vừa nhận; không cần tạo hai socket cho hai chiều.
- ▶ **Trong Python:** mô-đun socket của thư viện chuẩn — chỉ cần import socket, không phải pip install.

Python — socket đầu tiên

```
import socket

s = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)
print(s.family.name)    # AF_INET
print(s.type.name)      # SOCK_STREAM
print(s.fileno())       # mô tả tệp
s.close()
```

2.1 Một socket được định danh như thế nào?



giao thức	IP nguồn	cổng nguồn	IP đích	cổng đích
TCP	192.168.1.10	51234	203.0.113.5	443

bộ năm (five-tuple) — xác định duy nhất một kết nối trong toàn hệ thống

- ▶ **Socket đã kết nối:** được xác định bởi bộ năm ở trên; đổi một thành phần bất kỳ là một kết nối khác.
- ▶ **Socket lắng nghe:** chỉ cần bộ ba (giao thức, IP cục bộ, cổng cục bộ) — chưa có đầu bên kia.
- ▶ **Nhờ vậy:** hàng nghìn client cùng nối tới cổng 443 của một web server vẫn được phân biệt, vì cổng nguồn của mỗi client khác nhau.
- ▶ **Xem tận mắt:** netstat -an (Windows), netstat -an | grep 9000 hoặc lsof -i :9000 (macOS, Linux).

Hai trình duyệt trên cùng máy mở cùng một trang web: khác nhau ở cổng nguồn — đó là lý do server không nhầm lẫn.



Hàng số	Ý nghĩa	Dạng địa chỉ / giao thức	Dùng khi
AF_INET	Địa chỉ IPv4	(host, port)	Phổ biến nhất — dùng suốt học phần
AF_INET6	Địa chỉ IPv6	(host, port, flowinfo, scope_id)	Mạng IPv6, ứng dụng hiện đại
AF_UNIX	Socket cục bộ (Unix domain)	Đường dẫn tệp: "/tmp/app.sock"	Hai tiến trình trên cùng máy (macOS, Linux)
SOCK_STREAM	Luồng byte, tin cậy	→ giao thức TCP	Chat, truyền tệp, HTTP (Chương 3)
SOCK_DGRAM	Datagram, không kết nối	→ giao thức UDP	Hỏi – đáp ngắn, broadcast (Chương 4)
SOCK_RAW	Gói thô, tự dựng header	→ ICMP, IP	Ping, công cụ mạng; cần quyền quản trị

socket.socket(family=AF_INET, type=SOCK_STREAM, proto=0) — hai tham số đầu quyết định "loại" socket; proto hầu như luôn để 0 cho hệ điều hành tự chọn.



2.1

Trình tự lời gọi socket: TCP và UDP

TCP — hướng kết nối

SERVER

`socket()`

`bind()`

`listen()`

`accept()`

`recv()` / `send()`

`close()`

CLIENT

`socket()`

`connect()`

`send()` / `recv()`

`close()`

UDP — không kết nối

SERVER

`socket()`

`bind()`

`recvfrom()`

`sendto()`

`close()`

CLIENT

`socket()`

`sendto()`

`recvfrom()`

`close()`

TCP phải thiết lập kết nối trước (`bind` → `listen` → `accept` ↔ `connect`); UDP gửi thẳng datagram kèm địa chỉ. Chương 3 và Chương 4 sẽ đi sâu từng mô hình.



SERVER — socket lắng nghe

```
s = socket(); bind("", 9000);  
listen(5)
```

accept() → sinh socket mới

conn₁ ('192.168.1.21', 51234) **Client A**

conn₂ ('192.168.1.22', 49876) **Client B**

conn₃ ('192.168.1.23', 60011) **Client C**

- ▶ **Socket lắng nghe:** tạo bằng bind() và listen(); chỉ làm một việc — nhận yêu cầu kết nối. Không dùng để gửi hay nhận dữ liệu.
- ▶ **Mỗi lần accept():** hệ điều hành sinh ra một socket phiên riêng cho client vừa nối: conn, addr = s.accept().
- ▶ **Trao đổi dữ liệu:** chỉ thực hiện trên conn; s vẫn tiếp tục lắng nghe để đón client kế tiếp.
- ▶ **Đóng đúng chỗ:** conn.close() kết thúc một phiên; s.close() dừng hẳn dịch vụ.
- ▶ **Lỗi kinh điển:** gọi s.recv() trên socket lắng nghe sẽ nhận OSError; phải gọi conn.recv().



2.1

Chặn (blocking) và thời gian chờ

- ▶ **Mặc định là chặn:** `accept()`, `connect()`, `recv()`, `sendall()` dừng chương trình cho tới khi xong việc — đơn giản nhưng mỗi lúc chỉ làm được một việc.
- ▶ **Hệ quả:** server tuần tự "đứng im" ở `accept()`; client treo ở `recv()` nếu server không trả lời; giao diện tkinter bị đơ.
- ▶ **Đặt hạn:** `s.settimeout(giây)` — quá hạn thì ném `TimeoutError`; `settimeout(None)` quay lại chế độ chặn.
- ▶ **Không chặn:** `s.setblocking(False)` trả về ngay, kèm `BlockingIOError` nếu chưa có dữ liệu — nền của selectors (Chương 6).
- ▶ **Cách dùng thật:** ứng dụng nhiều client dùng luồng (Chương 7) hoặc `asyncio` (Chương 6) thay vì chờ suông.

Python — Đặt thời gian chờ

```
import socket

s = socket.socket()
s.settimeout(5.0)      # 5 giây
try:
    s.connect(("10.0.0.99", 9000))
    data = s.recv(1024)
except TimeoutError:
    print("Quá hạn, bỏ qua")
except ConnectionRefusedError:
    print("Server chưa chạy")
finally:
    s.close()

# s.settimeout(None) -> chặn lại
# s.setblocking(False) -> không chặn
```



2.1

Điều gì thực sự xảy ra khi send() và recv()?

- ▶ **send(data)**: chỉ chép dữ liệu vào bộ đệm gửi của hệ điều hành rồi trả về số byte đã chép — có thể ít hơn len(data).
- ▶ **sendall(data)**: lặp lại cho tới khi gửi hết. Trong học phần hãy luôn dùng sendall().
- ▶ **recv(n)**: lấy tối đa n byte đang có trong bộ đệm nhận; có thể trả về ít hơn n và chỉ trả về khi đã có ít nhất 1 byte.
- ▶ **recv() trả b""**: bên kia đã đóng kết nối (EOF) — đây là dấu hiệu để thoát vòng lặp.
- ▶ **Hệ quả quan trọng**: TCP không giữ ranh giới thông điệp — hai lần send() có thể về trong một lần recv() và ngược lại (Chương 3).

Python — gửi và nhận đúng cách

```
# Gửi: luôn dùng sendall
msg = "Xin chào\n".encode("utf-8")
s.sendall(msg)

# Nhận: kiểm tra b"" trước khi dùng
chunk = s.recv(1024)
if not chunk:
    print("Đối phương đã đóng")
else:
    print(chunk.decode("utf-8"))

# n nên là lũy thừa của 2:
# 1024, 2048, 4096
```

Bộ đệm gửi và bộ đệm nhận nằm trong nhân hệ điều hành — send() nhanh không có nghĩa dữ liệu đã tới nơi.



Nhóm cổng	Dải	Ai dùng	Ghi chú cho sinh viên
Well-known	0 – 1023	HTTP 80, HTTPS 443, SSH 22, DNS 53	Phải có quyền quản trị mới bind() được
Registered	1024 – 49151	MySQL 3306, ứng dụng tự viết	Dải nên dùng: 5000, 8080, 9000
Dynamic / ephemeral	49152 – 65535	Cổng tạm hệ điều hành cấp cho client	bind((" ", 0)) để hệ điều hành tự chọn

✗ Address already in use (Errno 48 macOS, 98 Linux, 10048 Windows)

Cổng vừa dùng còn ở trạng thái TIME_WAIT vài chục giây. Đặt `s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` TRƯỚC `bind()` để khởi động lại ngay.

Kiểm tra trước khi chạy: `netstat -an | grep 9000 (macOS, Linux)` hoặc `netstat -ano | findstr 9000 (Windows)`.



Tóm tắt 2.1 — Giới thiệu Socket



1. Socket là điểm cuối của kênh liên lạc hai chiều do hệ điều hành cung cấp; lập trình mạng là lập trình với socket API.
2. Một kết nối được xác định duy nhất bởi bộ năm: giao thức, IP nguồn, cổng nguồn, IP đích, cổng đích.
3. Chọn socket bằng hai hằng số: họ địa chỉ (AF_INET, AF_INET6, AF_UNIX) và kiểu (SOCK_STREAM → TCP, SOCK_DGRAM → UDP).
4. TCP: socket → bind → listen → accept ↔ connect → send/recv → close. UDP: socket → bind → recvfrom/sendto → close.
5. accept() sinh ra socket phiên riêng cho mỗi client; socket lắng nghe không dùng để truyền dữ liệu.
6. Mọi lời gọi mặc định là chặn; send() chỉ chép vào bộ đệm, recv() trả b"" khi bên kia đóng; cổng nên chọn từ 1024 trở lên.



2.2

Mô-đun socket trong Python

Bên trong mô-đun socket; tạo socket và địa chỉ; bind, listen, accept, connect; gửi và nhận bytes; close và shutdown; echo server – echo client; tùy chọn và lỗi thường gặp





Nhóm	Thành phần tiêu biểu	Vai trò
Lớp chính	socket.socket	Đối tượng socket với đầy đủ phương thức mạng
Hằng số họ / kiểu	AF_INET, AF_INET6, SOCK_STREAM, SOCK_DGRAM	Chọn loại socket khi tạo
Hằng số tùy chọn	SOL_SOCKET, SO_REUSEADDR, TCP_NODELAY	Tinh chỉnh hành vi qua setsockopt()
Hàm tiện ích	create_server(), create_connection()	Gộp nhiều bước thành một lời gọi (C5)
Hàm tên miền	gethostname(), gethostbyname(), getaddrinfo()	Phân giải tên ↔ địa chỉ (mục 2.3)
Hàm chuyển đổi	htons(), ntohs(), inet_pton(), inet_ntop()	Thứ tự byte mạng, chuyển dạng địa chỉ
Ngoại lệ	OSError và lớp con: ConnectionRefusedError, gaierror	Bất lỗi khi lập trình mạng

Tất cả nằm trong thư viện chuẩn: import socket là đủ. Tra cứu đầy đủ tại docs.python.org/3/library/socket.html



2.2 Tạo đối tượng socket

- **Cú pháp:** `socket.socket(family=AF_INET, type=SOCK_STREAM, proto=0)` — bỏ trống hết thì mặc định là TCP trên IPv4.
- **Kết quả:** một đối tượng bọc lấy mô tả tệp của hệ điều hành; `s.family`, `s.type`, `s.fileno()` cho biết thông tin.
- **Luôn đóng:** dùng câu lệnh `with` để socket tự đóng kể cả khi có lỗi — tránh rò rỉ tài nguyên và kẹt cổng.
- **Chỉ tạo thôi chưa đủ:** socket vừa tạo chưa gắn địa chỉ, chưa kết nối; phải `bind()` và `listen()` (server) hoặc `connect()` (client).

Python — tạo và đóng socket

```
import socket
```

```
# Cách 1 — tự nhớ mà đóng
s = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)
# ... dùng s ...
s.close()
```

```
# Cách 2 — nên dùng: with
with socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM) as u:
    print(u.family.name) # AF_INET
    print(u.type.name)  # SOCK_DGRAM
# ra khỏi with -> u.close()
```



2.2

Địa chỉ trong Python: bộ (host, port)

- ▶ **Với AF_INET:** địa chỉ là một bộ hai phần tử (host, port) — host là chuỗi, port là số nguyên 0 – 65535.
- ▶ **host phía server:** "" hoặc "0.0.0.0" là mọi card mạng; "127.0.0.1" chỉ trong máy; "192.168.1.10" một card cụ thể.
- ▶ **host phía client:** tên miền ("example.com") hoặc địa chỉ IP; Python tự phân giải tên qua DNS (mục 2.3).
- ▶ **Với AF_INET6:** bộ bốn (host, port, flowinfo, scope_id); "::" tương ứng với "0.0.0.0" của IPv4.
- ▶ **Xem lại địa chỉ:** getsockname() cho biết đầu này, getpeername() cho biết đầu kia của socket.

Python — các dạng địa chỉ

```
# Server: lắng nghe mọi card mạng
srv = ("", 9000)

# Chỉ trong máy này
loc = ("127.0.0.1", 9000)

# Client: theo tên miền
web = ("example.com", 80)

# Để hệ điều hành tự chọn cổng
s.bind("", 0)
print(s.getsockname())
# ('0.0.0.0', 54321)
print(c.getpeername())
# ('93.184.216.34', 80)
```

Server bind "127.0.0.1" thì máy khác không nối vào được — lỗi rất hay gặp khi thực hành hai máy.



2.2

bind() và listen(): dựng phía server

- ▶ **bind(address)**: gắn socket vào (IP, cổng) cục bộ. Chỉ server bắt buộc gọi; client bỏ qua để hệ điều hành cấp cổng tạm.
- ▶ **listen(backlog)**: chuyển socket sang trạng thái lắng nghe và mở hàng đợi chờ; backlog là số kết nối chờ tối đa (thường đặt 5).
- ▶ **Thứ tự bắt buộc**: socket() → bind() → listen() → accept(); làm sai thứ tự sẽ nhận OSError.
- ▶ **SO_REUSEADDR**: đặt trước bind() để khởi động lại server ngay mà không gặp "Address already in use".
- ▶ **Hàng đợi đầy**: client đến khi hàng đợi đã đầy sẽ bị từ chối hoặc phải chờ — lý do phải phục vụ nhanh rồi quay lại accept().

Python — mở cổng lắng nghe

```
import socket

s = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET,
             socket.SO_REUSEADDR, 1)
s.bind("", 9000)
s.listen(5)
print("Đang lắng nghe cổng 9000...")

# Kiểm tra ở terminal:
# netstat -an | grep 9000
```



2.2

accept(): chấp nhận kết nối

- ▶ **conn, addr = s.accept():** chặn cho tới khi có client kết nối; trả về socket phiên conn và địa chỉ addr = (IP, cổng) của client.
- ▶ **Từ đây trở đi:** mọi lệnh đọc và ghi thực hiện trên conn, không phải trên s.
- ▶ **Phục vụ nhiều client:** bọc accept() trong while True; muốn phục vụ đồng thời phải dùng luồng hoặc asyncio (Chương 6 – 7).
- ▶ **conn cũng là socket:** có đủ recv(), sendall(), getpeername(), settimeout(), close(); nên viết with conn: để tự đóng.

Python — vòng lặp accept

```
while True:
    conn, addr = s.accept()
    print("Client:", addr)
    # ('127.0.0.1', 51234)
    with conn:
        data = conn.recv(1024)
        if data:
            conn.sendall(data)
    # hết with -> conn.close()
    # rồi quay lại accept()
```

Server tuần tự: chỉ phục vụ xong client này mới đón client kế tiếp.



2.2 connect(): phía client

- ▶ **connect((host, port))**: khởi tạo bắt tay ba bước tới server; thành công thì socket sẵn sàng gửi và nhận.
- ▶ **Không cần bind()**: hệ điều hành tự cấp một cổng tạm cho client — đó là cổng nguồn trong bộ năm.
- ▶ **Lỗi hay gặp**: `ConnectionRefusedError` (server chưa chạy, sai cổng), `TimeoutError` (sai IP, tường lửa chặn), `socket.gaierror` (sai tên miền).
- ▶ **connect_ex()**: trả về mã lỗi thay vì ném ngoại lệ — tiện khi thử nhiều địa chỉ hoặc quét cổng.
- ▶ **Server phải chạy trước**: không có server lắng nghe thì `connect()` thất bại ngay lập tức.

Python — client kết nối

```
import socket

s = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)

try:
    s.connect(("127.0.0.1", 9000))
    msg = "Xin chào".encode("utf-8")
    s.sendall(msg)
    reply = s.recv(1024)
    print(reply.decode("utf-8"))
except ConnectionRefusedError:
    print("Server chưa khởi động")
finally:
    s.close()
```




2.2

Gửi và nhận dữ liệu: chỉ có bytes

- ▶ **Chỉ nhận bytes:** `s.sendall("abc")` báo `TypeError`; phải `"abc".encode("utf-8")`; nhận về thì `.decode("utf-8")`.
- ▶ **`sendall(b)` và `send(b)`:** `sendall` gửi hết; `send` trả về số byte thực gửi — chỉ dùng khi tự quản lý bộ đệm.
- ▶ **`recv(n)`:** đọc tối đa `n` byte; `n` thường là 1024 hoặc 4096. Trả về ít hơn `n` là chuyện bình thường.
- ▶ **Nhận cho đủ:** lặp `recv()` và nối bytes cho tới khi gặp `b""` (EOF) hoặc đủ độ dài đã thỏa thuận trong giao thức.
- ▶ **Tiếng Việt:** luôn dùng UTF-8 ở cả hai đầu; một ký tự có dấu chiếm nhiều byte nên `len(chuỗi)` khác `len(bytes)`.

Python — nhận cho tới khi hết

```
def recv_all(sock):
    buf = b""
    while True:
        chunk = sock.recv(4096)
        if not chunk: # b"" -> EOF
            break
        buf += chunk
    return buf

msg = recv_all(conn).decode("utf-8")

txt = "Chào"
print(len(txt)) # 4 ký tự
print(len(txt.encode())) # 6 byte
```



- ▶ **close():** trả socket về hệ điều hành; sau đó mọi thao tác đều lỗi. Luôn đóng — bằng with hoặc finally.
- ▶ **shutdown(how):** đóng một chiều — SHUT_WR báo "tôi hết gửi" (bên kia recv() nhận b""), SHUT_RD, SHUT_RDWR.
- ▶ **Vì sao cần shutdown:** khi client gửi xong và chờ toàn bộ trả lời, close() ngay có thể làm mất dữ liệu chưa đọc.
- ▶ **Bắt TIME_WAIT:** cổng server còn bị giữ vài chục giây sau khi đóng — SO_REUSEADDR giúp khởi động lại ngay.

Python — báo "đã gửi xong"

```
# Báo cho đầu kia: tôi hết gửi
s.sendall(request)
s.shutdown(socket.SHUT_WR)
```

```
reply = b""
while True:
    chunk = s.recv(4096)
    if not chunk:
        break
    reply += chunk
s.close()

# SHUT_RD    - thôi nhận
# SHUT_RDWR - đóng cả hai chiều
```



2.2

Ví dụ 1: Echo server TCP

```
echo_server.py — python echo_server.py

import socket
HOST, PORT = "", 9000

with socket.socket(socket.AF_INET,
                   socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET,
                 socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(5)
    print(f"Server chờ tại cổng {PORT}...")

    while True:
        conn, addr = s.accept()
        print("Đã nối:", addr)
        with conn:
            while True:
                data = conn.recv(1024)
                if not data:
                    break
                conn.sendall(data)
```

Đọc mã theo 6 bước

1

Tạo socket TCP/IPv4 trong khối with để chắc chắn được đóng.

2

Bật SO_REUSEADDR để chạy lại ngay sau khi tắt.

3

bind(("", 9000)): lắng nghe trên mọi card mạng.

4

listen(5): mở hàng đợi, sẵn sàng đón kết nối.

5

Vòng ngoài: accept() lấy conn của từng client.

6

Vòng trong: recv → gặp b"" thì thoát, ngược lại sendall trả lại nguyên văn.

Echo = trả lại đúng những gì nhận được. Đây là "Hello World" của lập trình mạng.



2.2 Ví dụ 2: Echo client TCP

```
echo_client.py — python echo_client.py

import socket
HOST, PORT = "127.0.0.1", 9000

with socket.socket(socket.AF_INET,
                   socket.SOCK_STREAM) as s:
    s.settimeout(5.0)
    try:
        s.connect((HOST, PORT))
    except OSError as e:
        raise SystemExit(f"Không nối được: {e}")
    print("Đã nối tới", s.getpeername())
    while True:
        text = input("Nhập (Enter để thoát): ")
        if not text:
            break
        s.sendall(text.encode("utf-8"))
        data = s.recv(1024)
        if not data:
            print("Server đã đóng"); break
        print("Trả lời:", data.decode("utf-8"))
```

Những chỗ cần chú ý

- ▶ **settimeout(5.0)**: không để chương trình treo vô hạn khi server im lặng.
- ▶ **try quanh connect()**: thoát lịch sự kèm thông báo để hiểu thay vì đổ vệt lỗi.
- ▶ **encode / decode**: socket chỉ hiểu bytes; UTF-8 để giữ dấu tiếng Việt.
- ▶ **Kiểm tra b""**: server có thể đóng bất cứ lúc nào.
- ▶ **Chạy hai máy**: đổi HOST thành IP thật của máy chạy server.



2.2 Chạy thử và quan sát

Terminal — ba cửa sổ

```
# Cửa sổ 1 — server
$ python echo_server.py
Server chờ tại cổng 9000...
Đã nối: ('127.0.0.1', 51234)

# Cửa sổ 2 — client
$ python echo_client.py
Đã nối tới ('127.0.0.1', 9000)
Nhập (Enter để thoát): Xin chào
Trả lời: Xin chào

# Cửa sổ 3 — kiểm tra và thử tay
$ netstat -an | grep 9000
tcp4  0  0  *.9000  *.*      LISTEN
$ ncat 127.0.0.1 9000
test -> test
```

- ▶ **Hai máy khác nhau:** client đổi HOST thành IP thật của máy chạy server; server vẫn bind("").
- ▶ **Tường lửa:** Windows hỏi quyền lần đầu — chọn cho phép ở mạng riêng (Private network).
- ▶ **ncat / telnet:** đóng vai client để thử server trước khi viết mã.
- ▶ **Wireshark:** lọc tcp.port == 9000 để nhìn thấy bắt tay ba bước và nội dung thông điệp.
- ▶ **Ctrl + C:** dừng server; nếu báo cổng bận, chờ vài chục giây hoặc bật SO_REUSEADDR.

Mẹo: Đặt hai cửa sổ terminal cạnh nhau để nhìn thấy đồng thời cả hai đầu của kết nối.



2.2

Socket UDP tối giản

```
udp_server.py
import socket

s = socket.socket(socket.AF_INET,
                  socket.SOCK_DGRAM)
s.bind(("", 9001))
print("UDP server tại cổng 9001")

while True:
    data, addr = s.recvfrom(1024)
    print(addr, data.decode("utf-8"))
    s.sendto(data, addr)
```

```
udp_client.py
import socket

s = socket.socket(socket.AF_INET,
                  socket.SOCK_DGRAM)
s.settimeout(3.0)
s.sendto("Xin chào".encode("utf-8"),
        ("127.0.0.1", 9001))

data, addr = s.recvfrom(1024)
print("Từ", addr, ":", data.decode("utf-8"))
s.close()
```

Không listen(), không accept(), không connect(): mỗi datagram tự mang địa chỉ. **Đổi lại, không có gì bảo đảm gói tin tới nơi — chi tiết ở Chương 4.**



Tùy chọn	Mức	Ý nghĩa	Khi nào dùng
SO_REUSEADDR	SOL_SOCKET	Cho phép bind lại cổng còn TIME_WAIT	Mọi server trong học phần
SO_BROADCAST	SOL_SOCKET	Cho phép gửi datagram quảng bá	UDP broadcast (Chương 4)
SO_KEEPALIVE	SOL_SOCKET	Tự dò kết nối đã chết	Kết nối giữ lâu: chat, giám sát
SO_RCVBUF / SO_SNDBUF	SOL_SOCKET	Kích thước bộ đệm nhận / gửi	Tinh chỉnh hiệu năng khi truyền lớn
TCP_NODELAY	IPPROTO_TCP	Tắt thuật toán Nagle, gửi ngay	Thông điệp nhỏ, cần độ trễ thấp

```
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)    # bật
print(s.getsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF))    # đọc kích thước bộ đệm nhận
```



Phương thức	Vai trò	Bên sử dụng
<code>bind((host, port))</code>	Gắn địa chỉ cục bộ cho socket	Server (TCP, UDP)
<code>listen(backlog)</code>	Mở hàng đợi, chuyển sang trạng thái lắng nghe	Server TCP
<code>accept() → (conn, addr)</code>	Chấp nhận một kết nối, sinh socket phiên	Server TCP
<code>connect((host, port))</code>	Kết nối tới server (bắt tay ba bước)	Client TCP
<code>send(b) / sendall(b)</code>	Gửi bytes; sendall gửi cho bằng hết	Cả hai (TCP)
<code>recv(n)</code>	Nhận tối đa n byte; b"" nghĩa là bên kia đã đóng	Cả hai (TCP)
<code>sendto(b, addr) / recvfrom(n)</code>	Gửi / nhận datagram kèm địa chỉ	Cả hai (UDP)
<code>settimeout(s) / setblocking(f)</code>	Đặt thời gian chờ, chọn chế độ chặn	Cả hai
<code>getsockname() / getpeername()</code>	Địa chỉ đầu này / đầu kia của socket	Cả hai
<code>close() / shutdown(how)</code>	Đóng hẳn / đóng một chiều	Cả hai

Mọi phương thức trên đều có thể ném OSError — luôn nghĩ tới try/except khi lập trình mạng.



Ngoại lệ / thông báo	Nguyên nhân thường gặp	Cách xử lý
<code>ConnectionRefusedError</code>	Server chưa chạy hoặc sai số hiệu cổng	Khởi động server trước; kiểm tra bằng netstat
<code>TimeoutError</code>	Sai IP, tường lửa chặn, mạng nghẽn	Đặt <code>settimeout()</code> hợp lý, bắt ngoại lệ, thử ping
<code>OSError: Address already in use</code>	Cổng đang bị chiếm hoặc còn <code>TIME_WAIT</code>	Bật <code>SO_REUSEADDR</code> , đổi cổng, tắt tiến trình cũ
<code>OSError: Permission denied</code>	<code>bind()</code> cổng < 1024 khi không có quyền quản trị	Dùng cổng từ 1024 trở lên: 8080, 9000
<code>socket.gaierror</code>	Sai tên miền hoặc DNS không trả lời	Kiểm tra chính tả, thử nslookup (mục 2.3)
<code>ConnectionResetError / BrokenPipeError</code>	Đầu bên kia đóng đột ngột giữa chừng	Bắt ngoại lệ, dọn dẹp rồi đóng socket
<code>TypeError: a bytes-like object is required</code>	Gửi str thay vì bytes	Gọi <code>.encode("utf-8")</code> trước khi gửi

Tất cả đều là lớp con của `OSError`: `except OSError as e` bắt được mọi trường hợp — nhưng bắt riêng thì thông báo cho người dùng rõ hơn.



Tóm tắt 2.2 — Mô-đun socket trong Python



1. `import socket` là đủ: mô-đun có lớp `socket`, các hằng số `AF_/SOCK_/SO_`, hàm tên miền và các ngoại lệ.
2. Server: `socket()` → `setsockopt(SO_REUSEADDR)` → `bind((host, port))` → `listen(backlog)` → `accept()` trong vòng lặp.
3. Client: `socket()` → `connect((host, port))`; không cần bind vì hệ điều hành cấp cổng tạm.
4. Truyền dữ liệu chỉ bằng bytes: `sendall(s.encode("utf-8"))` và `recv(n).decode("utf-8")`; `recv` trả `b""` là bên kia đã đóng.
5. Dùng `with` để socket luôn được đóng; `shutdown(SHUT_WR)` để báo "đã gửi xong" mà vẫn đọc tiếp được trả lời.
6. Echo server và echo client là bộ khung để phát triển thành ứng dụng chat ở các chương sau.



2.3

Phân giải tên miền DNS

*Vì sao cần DNS; cơ chế phân giải; gethostname và
gethostbyname; getaddrinfo; client chạy được cả IPv4 lẫn
IPv6; tra ngược và xử lý lỗi*





DNS (Domain Name System) là hệ thống phân tán ánh xạ tên miền dễ nhớ (www.python.org) sang địa chỉ IP mà socket thực sự cần (151.101.132.223).

- ▶ **Socket chỉ hiểu IP:** `connect(("example.com", 80))` chạy được là vì Python đã âm thầm hỏi DNS trước khi kết nối.
- ▶ **Vì sao dùng tên:** dễ nhớ; đổi máy chủ không phải sửa client; một tên trở nhiều IP để cân bằng tải và dự phòng.
- ▶ **Giao thức:** DNS chạy trên UDP cổng 53 (chuyển sang TCP khi câu trả lời dài), có bộ nhớ đệm nhiều tầng.
- ▶ **Trong Python:** các hàm ở mức mô-đun — `gethostbyname`, `gethostbyname`, `getaddrinfo`, `gethostbyaddr`.

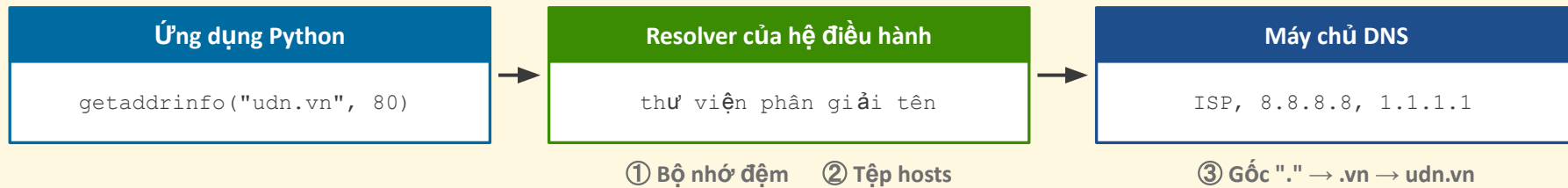
Python — tên miền → IP

```
import socket
```

```
ip = socket.gethostbyname(  
    "www.python.org")
```

```
print(ip)  
# 151.101.132.223
```

```
# Có thể đổi theo thời gian  
# và theo vị trí địa lý
```



- ▶ ① **Bộ nhớ đệm**: câu trả lời gần đây được giữ lại theo TTL, nên lần tra thứ hai rất nhanh.
- ▶ ② **Tập hosts**: C:\Windows\System32\drivers\etc\hosts (Windows) hoặc /etc/hosts (macOS, Linux) — được ưu tiên trước DNS, rất tiện để thử nghiệm.
- ▶ ③ **Truy vấn đệ quy**: nếu chưa biết, máy chủ DNS hỏi lần lượt gốc "." → .vn → udn.vn cho tới khi có câu trả lời.
- ▶ **Cấu hình DNS**: theo card mạng hoặc do DHCP cấp; có thể đổi sang 8.8.8.8 (Google) hay 1.1.1.1 (Cloudflare).
- ▶ **Kiểm tra bằng công cụ**: nslookup www.python.org, dig +short www.python.org, hoặc ping tên miền.



2.3

gethostname() và gethostbyname()

- ▶ **gethostname()**: trả về tên của chính máy đang chạy chương trình.
- ▶ **gethostbyname(name)**: tên → một địa chỉ IPv4 duy nhất, dạng chuỗi. Ngắn gọn, dễ dùng cho bài tập.
- ▶ **Chỉ IPv4**: hàm này không trả về IPv6 — ứng dụng thực tế nên dùng getaddrinfo().
- ▶ **gethostbyname_ex(name)**: trả về bộ ba (tên chính, danh sách bí danh, danh sách IPv4) — thấy được nhiều địa chỉ của một tên.
- ▶ **Lấy IP thật của máy**: gethostbyname(gethostname()) có thể ra 127.0.0.1 trên Linux; cách chắc chắn hơn là mở socket UDP tới 8.8.8.8 rồi getsockname().

Python — tên máy và địa chỉ IP

```
import socket

print(socket.gethostname())
# LAB-PC07

name = "www.python.org"
print(socket.gethostbyname(name))
# 151.101.132.223

print(socket.gethostbyname_ex(name))
# ('www.python.org', [], ['151...'])

# IP thật của máy trong mạng LAN
u = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM)
u.connect(("8.8.8.8", 80))
print(u.getsockname()[0])
# 192.168.1.10 (không gửi gói nào)
u.close()
```



2.3

getaddrinfo(): cách tra cứu hiện đại

- ▶ **Một hàm cho mọi trường hợp:** trả về danh sách các bộ 5 (family, type, proto, canonname, sockaddr) — dùng được ngay cho socket() và connect().
- ▶ **Cả IPv4 lẫn IPv6:** kết quả được hệ điều hành sắp xếp theo thứ tự ưu tiên, nên cứ thử lần lượt từ đầu danh sách.
- ▶ **Tham số:** getaddrinfo(host, port, family=0, type=0, proto=0, flags=0); port có thể là số hoặc tên dịch vụ ("http").
- ▶ **Nên lọc:** truyền type=SOCK_STREAM để chỉ lấy TCP, family=AF_INET nếu chỉ muốn IPv4 — tránh danh sách trùng lặp.

Python — getaddrinfo

```
import socket

for res in socket.getaddrinfo(
    "www.python.org", 80,
    type=socket.SOCK_STREAM):
    fam, typ, _, _, addr = res
    print(fam.name, addr)

# AF_INET6 ('2a04:4e42::223', 80,
#          0, 0)
# AF_INET ('151.101.132.223', 80)

# Tên dịch vụ thay cho số cổng:
# getaddrinfo("python.org", "http")
```



Python — thử lần lượt các địa chỉ

```
import socket
def connect_any(host, port, timeout=5):
    last = None
    for fam, typ, proto, _, addr in \
        socket.getaddrinfo(
            host, port,
            type=socket.SOCK_STREAM):
        s = socket.socket(fam, typ, proto)
        try:
            s.settimeout(timeout)
            s.connect(addr)
            return s
        except OSError as e:
            s.close()
            last = e
    raise last

# Thư viện chuẩn đã làm sẵn việc này:
s = socket.create_connection(
    ("example.com", 80), timeout=5)
```

- ▶ **Vấn đề:** máy chủ có thể chỉ có IPv6, hoặc đường IPv4 hỏng — client cứng nhắc AF_INET sẽ thất bại.
- ▶ **Cách làm chuẩn:** duyệt danh sách của `getaddrinfo()`, thử tạo socket và `connect()` cho tới khi thành công.
- ▶ **Lỗi tắt:** `socket.create_connection((host, port), timeout)` đã gói sẵn toàn bộ vòng lặp này (Chương 5).
- ▶ **Phía server:** `socket.create_server(("", 9000), family=AF_INET6, dualstack_ipv6=True)` phục vụ được cả hai họ địa chỉ.



Hàm	Trả về	Ví dụ
<code>gethostbyaddr(ip)</code>	Bộ (tên chính, bí danh, danh sách IP) — tra ngược IP → tên	<code>gethostbyaddr("8.8.8.8") → ("dns.google", ...)</code>
<code>getfqdn(name)</code>	Tên miền đầy đủ (fully qualified domain name)	<code>getfqdn("www") → "www.udn.vn"</code>
<code>getnameinfo(addr, flags)</code>	Bộ (tên máy, tên dịch vụ) từ một sockaddr	<code>getnameinfo(("8.8.8.8", 53), 0) → ("dns.google", ...)</code>
<code>getservbyname(name, proto)</code>	Số hiệu cổng của một dịch vụ chuẩn	<code>getservbyname("http", "tcp") → 80</code>
<code>getservbyport(port, proto)</code>	Tên dịch vụ ứng với số hiệu cổng	<code>getservbyport(53, "udp") → "domain"</code>
<code>inet_pton / inet_ntop</code>	Chuyển địa chỉ giữa dạng chuỗi và dạng bytes	<code>inet_pton(AF_INET, "1.2.3.4")</code>
<code>htons / ntohs / htonl / ntohl</code>	Đổi thứ tự byte giữa máy và mạng (big-endian)	<code>htons(9000)</code>

Tra ngược không phải lúc nào cũng thành công: nhiều địa chỉ IP không có bản ghi PTR nên `gethostbyaddr` sẽ ném `gaierror`.



2.3

Lỗi khi phân giải tên và cách xử lý

- ▶ **socket.gaierror:** lỗi phân giải tên — sai chính tả, mất mạng, hoặc máy chủ DNS không trả lời. Là lớp con của OSError.
- ▶ **Mã hay gặp:** EAI_NONAME (tên không tồn tại), EAI_AGAIN (DNS tạm thời không phản hồi — thử lại sau).
- ▶ **Chậm bất thường:** mỗi lần tra tên có thể mất hàng trăm mili-giây; trong vòng lặp hãy tra một lần rồi dùng lại địa chỉ IP.
- ▶ **Là lời gọi chặn:** getaddrinfo() chặn chương trình; ứng dụng có giao diện phải đặt nó ở luồng riêng (Chương 7) hoặc dùng asyncio (Chương 6).
- ▶ **Thử tay trước:** nslookup hoặc dig để phân biệt lỗi mạng với lỗi trong chương trình.

Python — bắt lỗi tên miền

```
import socket

try:
    ip = socket.gethostbyname(
        "khong-ton-tai.abc")
except socket.gaierror as e:
    print("Không phân giải:", e)
    # [Errno 8] nodename nor
    # servname provided
except OSError as e:
    print("Lỗi mạng:", e)
```

`gaierror = "get address info error".`



```
dns_lookup.py — python dns_lookup.py udn.vn

import socket, sys

name = sys.argv[1] if len(sys.argv) > 1 \
    else "www.python.org"

print("Máy này :", socket.gethostname())
print("Tra cứu :", name)

try:
    print("IPv4 :",
          socket.gethostbyname(name))
    for fam, _, _, _, addr in \
        socket.getaddrinfo(
            name, 80,
            type=socket.SOCK_STREAM):
        print(f" {fam.name:9} {addr[0]}")
    ip = socket.gethostbyname(name)
    print("Tra ngược:",
          socket.gethostbyaddr(ip)[0])
except socket.gaierror as e:
    print("Lỗi phân giải:", e)
```

Bài tập mở rộng

1

Nhận tham số cổng và in tên dịch vụ bằng `getservbyport()`.

2

In thêm bí danh của tên miền bằng `gethostbyname_ex()`.

3

Đo thời gian tra cứu lần đầu và lần thứ hai để thấy tác dụng của bộ nhớ đệm.

4

Đọc danh sách tên miền từ tệp và tra cứu lần lượt, bỏ qua tên lỗi.

5

So sánh kết quả với `nslookup` và giải thích khác biệt (nếu có).



Tóm tắt 2.3 — Phân giải tên miền DNS



1. Socket chỉ làm việc với địa chỉ IP; DNS ánh xạ tên miền dễ nhớ sang IP, chạy trên UDP cổng 53.
2. Máy tính phân giải theo thứ tự: bộ nhớ đệm → tệp hosts → máy chủ DNS được cấu hình → truy vấn đệ quy tới gốc.
3. `gethostname()` cho tên máy; `gethostbyname()` cho một IPv4; `gethostbyname_ex()` cho danh sách địa chỉ và bí danh.
4. `getaddrinfo()` là cách hiện đại: trả về danh sách bộ 5 dùng được ngay cho `socket()` và `connect()`, hỗ trợ cả IPv6.
5. Client bền vững duyệt lần lượt kết quả `getaddrinfo()`; `socket.create_connection()` đã gói sẵn cách làm này.
6. Lỗi phân giải là `socket.gaierror`; tra cứu là lời gọi chặn nên cần đặt ở luồng riêng trong ứng dụng có giao diện.



Tổng kết Chương 2 — Lập trình Socket với Python



2.1

Socket: khái niệm socket; bộ năm định danh kết nối; họ địa chỉ và kiểu socket; trình tự lời gọi TCP/UDP; socket lắng nghe và socket phiên; chặn, bộ đệm và chọn cổng.

2.2

Mô-đun socket: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `sendall()`, `recv()`, `close()`, `shutdown()`; `setsockopt(SO_REUSEADDR)`; echo server – echo client; bảng tra cứu và các lỗi thường gặp.

2.3

DNS: cơ chế phân giải tên; `gethostname()`, `gethostbyname()`, `gethostbyname_ex()`, `getaddrinfo()`, `gethostbyaddr()`; client chạy được cả IPv4 lẫn IPv6; xử lý `gaierror`.

→ Tiếp theo — Chương 3: Lập trình ứng dụng hướng kết nối TCP (Client – Server và ranh giới thông điệp)



Cảm ơn!

Chương 2 — Lập trình Socket với Python



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM

TÀI LIỆU THAM KHẢO

- [1] B. Rhodes, J. Goerzen — Foundations of Python Network Programming, 3rd ed., Apress, 2014 (giáo trình) — Chương 2, 3.
- [2] Đỗ Thị Hương Lan, Trần Hồng Nghi — Tài liệu hướng dẫn thực hành Lập trình mạng căn bản, NXB ĐHQG TP.HCM, 2024.
- [3] J. Kurose, K. Ross — Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021 — mục 2.7 Socket Programming.
- [4] Python Software Foundation — The Python Standard Library: socket — docs.python.org/3/library/socket.html
- [5] Real Python — Socket Programming in Python (Guide) — realpython.com/python-sockets/
- [6] RFC 1034, RFC 1035 (DNS); RFC 3493 (Basic Socket Interface Extensions for IPv6) — ietf.org.